Dr. Dobb's Journal December 2007

Avoid Calling Unknown Code While Inside a Critical Section

Don't walk blindly into a deadly embrace...

By Herb Sutter

Herb is a software architect at Microsoft and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

Our world is built on modular, composable software. We routinely expect that we don't need to know about the internal implementation details of a library or plug-in to be able to use it correctly.

The problem is that locks, and most other kinds of synchronization we use today, are not modular or composable. That is, given two separately authored modules, each of which is provably correct but uses a lock or similar synchronization internally, we generally can't combine them and know that the result is still correct and will not deadlock. There are only two ways to combine such a pair of lock-using modules safely:

- Option 1. Each module must know about the complete internal implementation of any function it calls in the other module (generally impossible for code you don't control).
- Option 2. Each module must be careful not to call into the other module while the calling code is inside a critical section (while holding a lock, for example).

Let's examine why Option 2 is generally the only viable choice, and what consequences it has for how we write concurrent code. For convenience, I'm going to cast the problem in terms of locks, but the general case arises whenever:

- The caller is inside one critical section.
- The callee directly or indirectly tries to enter another critical section, or performs another blocking call.
- Some other thread could try to enter the two critical sections in the opposite order.

Quick Recap: Deadlock

A deadlock (or deadly embrace) can happen anytime two different threads can try to acquire the same two locks (or more generally, acquire exclusive use of the resources protected by the same two synchronization objects) in opposite orders. Therefore, anytime you write code where a thread holds one lock *L1* and tries to acquire another lock *L2*, that code is liable to be one arm of a potential deadly embrace—unless you can eliminate the possibility that some other thread might try to acquire the locks in the other order.

We can use techniques such as lock hierarchies to guarantee that locks are taken in order. Unfortunately, those techniques do not compose either. For example, you can use a lock hierarchy and guarantee that the code you control follows the discipline, but you can't guarantee that other people's code will know anything about your lock levels, much less follow them correctly.

Example: Two Modules, One Lock Each

One fine day, you decide to write a new web browser that lets users write plug-ins to customize the behavior or rendering of individual page elements. Consider the following possible code, where we simply protect all the data structures representing elements on a given page using a single mutex *mutPage*:

```
// Example 1: Thread 1 of a potential deadly embrace
//
class CoreBrowser {
```

```
  ... other methods ...
  private void RenderElements() {
    mutPage.lock();  // acquire exclusion on the page elements
    try {
      for( each PageElement e on the page ) {
        DoRender( e );        // do our own default processing
        plugin.OnRender( e ); // let the plug-in have a crack at it
      }
    } finally {
      mutPage.unlock(); // and then release it
    }
  }
}
```

Do you see the potential for deadlock? The trouble is that if inside the call to *plugin.OnRender* the plug-in might acquire some internal lock of its own, which could be one arm of a potential deadly embrace. For example, consider this plug-in implementation that does some basic instrumentation of how many times certain actions have been performed and protects its internal data with a single mutex *mutMyData*:

```
class MyPlugin {
   ... other methods ...
  public void OnRender( PageElement e ) {
    mutMyData.lock();  // acquire exclusion on some internal shared da
    try {
      renderCount[e]++; // update #times e has been rendered
    } finally {
      mutMyData.unlock(); // and then release it
    }
  }
}
```

Thread 1 can therefore acquire *mutPage* and *mutMyData* in that order. Thread 1 is potential deadlock-bait, but the trouble will only manifest if some other Thread 2 that could run concurrently with the aforementioned code performs something like the following:

```
// Example 2: Thread 2 of a potential deadly embrace
//
class MyPlugin {
   ... other methods ...
  public void RefreshDisplay( PageElement e ) {
    mutMyData.lock();  // acquire exclusion on some internal shared da
    try {              // display stuff in a debug window
      for( each element e we've counted ) {
        listRenderedCount.Add( e.Name(), renderCount[e] );
      }
      textHiddenCount = browser.CountHiddenElements();
    } finally {
      mutMyData.unlock();  // and then release it
    }
  }
}
```

Notice how the plugin calls code unknown to it, namely *browser.CountHiddenElements*? You can probably see the trouble coming on like a steamroller:

```
class CoreBrowser {
  ... other methods ...
  public int CountHiddenElements() {
    mutPage.lock();  // acquire exclusion on the page elements
    try {
      int count = 0;
      for( each PageElement e on the page ) {
        if( e.Hidden() ) count++;
      }
      return count;
    } finally {
      mutPage.unlock(); // and then release it
    }
  }
}
```

Threads 1 and 2 can therefore acquire *mutPage* and *mutMyData* in the opposite order, so this is a deadlock waiting to happen if Threads 1 and 2 can ever run concurrently. For added fun, note that each mutex is purely an internal implementation detail of its module that is never exposed in the interface; neither module knows anything about the internal lock being used within the other. (Nor, in a better programming world than the one we now inhabit, should it have to.)

Example: Two Modules, But Only One Has Locks

Note that this kind of thing can happen even if both locks are in the same module, but control flow passes through another module so that you don't know what locks are taken. Consider the following modification, where the browser protects each page element using a separate mutex, which can be desirable to allow different parts of the page to be rendered concurrently:

```
// Example 3: Thread 1 of an alternative potential deadly embrace
//
class CoreBrowser {
  ... other methods ...
  private void RenderElements() {
    for( each PageElement e on the page ) {
      e.mut.lock();  // acquire exclusion on this page element
      try {
        DoRender( e ); // do our own default processing
        plugin.OnRender( e ); // let the plug-in have a
                              // crack at it
      } finally {
        e.mut.unlock(); // and then release it
      }
    }
  }
}
```

And consider a plug-in that does no locking of its own at all:

```
class MyPlugin {
  ... other methods ...
  public void OnRender( PageElement e ) {
    GuiCoord cPrev = browser.GetElemPosition( e.Previous() );
    GuiCoord cNext = browser.GetElemPosition( e.Next() );
    Use( e, cPrev, cNext ); // do something with the coords
  }
}
```

But which calls back into:

```
class CoreBrowser {
  ... other methods ...
  public GuiCoord GetElemPosition( PageElement e2 ) {
    e2.mut.lock();  // acquire exclusion on this page element
    try {
      return FigureOutPositionFor( e2 );
    } finally {
      e2.mut.unlock();  // and then release it
    }
  }
}
```

The order of mutex acquisition is:

```
for each element e
  acquire e.mut
    acquire e.Previous().mut
    release e.Previous().mut
    acquire e.Next().mut
    release e.Next().mut
  release e.mut
```

Perhaps the most obvious issue is that any pair of locks on adjacent elements can be taken in both orders by Thread 1; so this cannot possibly be part of a correct lock hierarchy discipline.

Because of the interference of the plug-in code, which does not even have any locks of its own, this code has a latent deadlock if any other concurrently running thread (including perhaps another instance of Thread 1 itself) can take any two adjacent elements' locks in any order. The deadlock-proneness is inherent in the design, which fails to guarantee a rigorous ordering of locks. In this respect, the original Example 1 was better, even though its locking was coarse-grained and less friendly to concurrent rendering of different page elements.

Consequences: What Is "Unknown Code"?

It's one thing to say "avoid calling unknown code while holding a lock" or while inside a similar kind of critical section. It's another to do it, because ther are so many ways to get into "someone else's code." Let's consider a few. While inside a critical section, including while holding a lock:

- **Avoid calling library functions.** A library function is the classic case of "someone else's code." Unless the library function is documented to not take any locks, deadlock problems can arise.
- **Avoid calling plug-ins.** Clearly, a plug-in is "someone else's code."
- **Avoid calling other callbacks, function pointers, functors, delegates, and so on.** C function pointers, C++ functors, C# delegates, and the like can also fairly obviously lead to "someone else's code." Sometimes, you know that a function pointer, functor, or delegate will always lead to your own code, and calling it is safe; but if you don't know that for certain, avoid calling it from inside a critical section.
- **Avoid calling virtual methods.** This may be less obvious and quite surprising, even Draconian; after all, virtual functions are common and pervasive. But every virtual function is an extensibility point that can lead to executing code that doesn't even exist today. Unless you control all derived classes (for example, the virtual function is internal to your module and not exposed for others to derive from), or you can somehow enforce or document that overrides must not take locks, deadlock problems can arise if it is called while inside a critical section.
- **Avoid calling generic methods, or methods on a generic type.** When writing a C++ template or a Java or .NET generic, we have yet another way to parameterize and intentionally let "someone else's code" be plugged into our own. Remember that anything you call on a generic type or method can be provided by anyone, and unless you know and control all the types with which your template or generic can be instantiated, avoid calling something generic from within a critical section.

Some of these restrictions may be obvious to you; others may be surprising at first.

Avoidance: Noncritical Calls

So you want to remove a call to unknown code from a critical section. But how? What can you do? Four options are: (a) move the call out of the critical section if you didn't need the exclusion anyway; (b) make copies of data inside the critical section and later pass the copies; (c) reduce the granularity or power of the critical section being held at the point of the call; or (d) instruct the callee sternly and hope for the best.

We can apply the first approach directly to Example 2. There is no reason the plugin needs to call *browser.CountHiddenElements*() while holding its internal lock. That call should simply be moved to before or after the critical section.

The second approach is to pass copies of data, which solves the correctness problem at the expense of space and performance. Variants of this approach include passing a subset of the data, and passing the copies via messages to run the callee asynchronously.

To improve Example 1, for instance, it might be appropriate to change the *RenderElements* method to hold the lock only long enough to take copies of the necessary shared information in a local container, then doing processing outside the lock, passing the copied elements. (This could be inappropriate if the data is very expensive to copy, or the callee needs to work on the real data.) Alternatively, perhaps the callee doesn't really need all the information it gets from being given direct access to the protected object, and it would be both sufficient and efficient to pass copies of just those parts of the data the callee does need.

The third option is to reduce the power or granularity of the critical section, which implicitly trades off ease-of-use because making your synchronization finer-tuned and/or finer-grained also makes it harder to code correctly. One example of reducing the power of the critical section is to replace a mutex with a reader-writer mutex so that multiple concurrent readers are allowed; if the only deadlocks could arise among threads that are only performing reads of the protected resources, then this can be a valid solution by enabling the use of a read-only lock instead of a read-write lock. And an example of making the critical section finer-grained is to replace a single mutex protecting a large data structure with mutexes protecting parts of the structure; if the only deadlocks possible are among threads that use different parts of the structure, then this can be a valid solution (Example 1 is not such a case).

The fourth option is to tell the callee not to block, which trades off enforceability. In particular, if you have the power to impose requirements on the callee (as you do with plug-ins to your software, but not with simple calls into existing third-party libraries), then you can require them to not take locks or otherwise perform blocking actions. Alas, these requirements are typically going to be limited to documentation, and are typically not enforceable automatically. Tell the callee what (not) to do, and hope he follows the yellow brick road.

Summary

Be aware of the many opportunities modern languages give us to call "someone else's code," and eliminate external opportunities for deadlock by not calling unknown code from within a critical section. If you additionally eliminate internal opportunities for deadlock by applying a lock hierarchy disciplin

within the code you control, your use of locks will be highly likely to be correct...and we'll consider lock hierarchies next month. Stay tuned.